

classifier_notebook

June 8, 2026

1 News Headline Classifier: Prompt Engineering Comparison

This notebook implements and compares three prompting strategies for news headline classification using Ollama LLMs.

1.1 1. Import Required Libraries

We start by importing the necessary libraries for building our prompt engineering classifier. These libraries help us: - **os** & **argparse**: Handle file operations and command-line arguments - **re**: Use regular expressions for pattern matching - **ollama**: Interact with local Ollama LLM models - **tabulate**: Format output in nice tables

```
[ ]: import os                                # Provides functions for interacting with the
      ↪operating system
import re                                    # Provides support for regular expressions
      ↪(pattern matching)
import argparse                             # Helps create command-line interfaces and parse
      ↪arguments
import ollama                               # Ollama Python API for interacting with local
      ↪LLMs
from tabulate import tabulate              # Used to print formatted tables in terminal
      ↪output
```

1.2 2. Define Few-Shot Examples and Categories

This section defines the core data structures used throughout our classifier:

- **FEW_SHOT_EXAMPLES**: 15 labeled headlines across 5 categories that we'll show to the model for few-shot learning. These help the model understand the classification patterns.
- **VALID_CATEGORIES**: The 5 news categories our classifier can predict: Politics, Technology, Sports, Finance, and Health
- **SYNTHETIC_DATASET**: A fallback dataset of 5 headlines with labels, used when real data isn't available

```
[ ]: FEW_SHOT_EXAMPLES = [
      # Politics examples
      {"text": "Senate Approves New Infrastructure Spending Bill.", "label":
      ↪"Politics"},
```

```

        {"text": "Prime Minister Announces Snap Election Amid Coalition Collapse.", "label": "Politics"},
        {"text": "Governor Signs Executive Order Restricting Industrial Emissions.", "label": "Politics"},

        # Technology examples
        {"text": "New Quantum Computing Chip Breakthrough Multiplies Processing Speeds.", "label": "Technology"},
        {"text": "Cybersecurity Breach Exposes Millions of User Accounts Worldwide.", "label": "Technology"},
        {"text": "Startup Launches Revolutionary Virtual Reality Headset for Remote Workers.", "label": "Technology"},

        # Sports examples
        {"text": "Underdog Team Secures Dramatic Victory in Championship Final.", "label": "Sports"},
        {"text": "Olympic Committee Announces New Host City for Upcoming Summer Games.", "label": "Sports"},
        {"text": "World Number One Tennis Star Withdraws From Tournament Due to Injury.", "label": "Sports"},

        # Finance examples
        {"text": "Stock Market Plummets as Tech Sector Experiences Massive Sell-Off.", "label": "Finance"},
        {"text": "Global Conglomerate Reports Record Profits in Q3 Financial Release.", "label": "Finance"},
        {"text": "Cryptocurrency Regulations Tighten Across European Markets.", "label": "Finance"},

        # Health examples
        {"text": "Study Finds Regular Exercise Significantly Reduces Risk of Heart Disease.", "label": "Health"},
        {"text": "Hospitals Face Severe Nurse Shortages Amid Seasonal Flu Surge.", "label": "Health"},
        {"text": "Researchers Map Human Genome Sequence to Uncover Rare Genetic Mutations.", "label": "Health"}
    ]

VALID_CATEGORIES = [
    "Politics",
    "Technology",
    "Sports",
    "Finance",
    "Health"
]
```

```

SYNTHETIC_DATASET = [
    {
        "headline": "Tech Giants Agree on New Open-Source AI Safety Standards",
        "label": "Technology"
    },
    {
        "headline": "Central Bank Raises Interest Rates by 25 Basis Points to Combat Inflation",
        "label": "Finance"
    },
    {
        "headline": "Star Striker Signs Record-Breaking Five-Year Contract Extension",
        "label": "Sports"
    },
    {
        "headline": "Parliament Votes to Pass Historic Climate Action Bill After Fierce Debate",
        "label": "Politics"
    },
    {
        "headline": "New FDA-Approved Breakthrough Drug Shows Promise in Halting Alzheimer's",
        "label": "Health"
    },
]

```

1.3 3. Label Cleaning Function

The `clean_label()` function is crucial for extracting valid category labels from raw LLM output.

Why is this needed? - Language models don't always output exactly what we ask for - They might include extra text, markdown, or multiple words - This function safely extracts the first valid category found using regex pattern matching

How it works: 1. Searches through all valid categories using regex with whole-word matching (\b) 2. Returns the first matching category (case-insensitive) 3. Returns "Unknown" if no valid category is found

```

[ ]: def clean_label(raw_output: str) -> str:

    # Loop through all valid categories
    for category in VALID_CATEGORIES:

```

```

# Use regex search to check if the category exists in output
# \b ensures whole-word matching
# re.IGNORECASE ignores capitalization differences
if re.search(rf"\b{category}\b", raw_output, re.IGNORECASE):

    # Return the matched valid category
    return category

# If no valid category found, return "Unknown"
return "Unknown"

```

1.4 4. Zero-Shot Classification

Zero-Shot Prompting is the simplest approach - the model receives ONLY task instructions with NO examples.

Key characteristics: - Minimal prompt tokens (most efficient) - Tests the model's ability to generalize from pre-training - Good baseline to compare other techniques against - Temperature set to 0.0 for deterministic/consistent outputs

How it works: 1. Define a system prompt explaining the task and valid categories 2. Send the headline to classify with no examples 3. Extract and clean the model's response 4. Return the prediction along with token counts for efficiency analysis

```

[ ]: def classify_zero_shot(headline: str, model_name: str) -> tuple[str, int, int]:

    # Create system prompt that instructs the model
    system_prompt = (

        # Tell the model its role
        "You are an expert news editor. "

        # Explain the task
        "Your task is to classify the provided headline "

        # Show allowed categories
        f"into exactly one of these categories: {VALID_CATEGORIES}. "

        # Strict output formatting instructions
        "Respond with ONLY the category name. "
        "Do not write markdown, intros, or punctuation."
    )

    # Send chat request to Ollama
    response = ollama.chat(

        # Specify which model to use

```

```

model=model_name,

# Conversation messages
messages=[

    # System role defines behavior/instructions
    {"role": "system", "content": system_prompt},

    # User message contains the actual headline
    {"role": "user", "content": f"Headline: {headline}"}
],

# Deterministic output
options={"temperature": 0.0}
)

# Extract generated text from response
raw_content = response['message']['content'].strip()

# Return:
# cleaned label,
# prompt token count,
# generated token count
return (
    clean_label(raw_content),
    response.get('prompt_eval_count', 0),
    response.get('eval_count', 0)
)

```

1.5 5. Few-Shot Classification

Few-Shot Prompting shows the model multiple labeled examples BEFORE asking it to classify a new headline.

Key characteristics: - Higher prompt tokens (includes all examples) - Uses in-context learning - the model learns patterns from examples - Often improves accuracy at the cost of more tokens - Still doesn't require fine-tuning the model - Temperature set to 0.0 for deterministic outputs

How it works: 1. Create a system prompt explaining the task 2. Format all 15 few-shot examples as: Headline: [text] (newline) Category: [label] 3. Append the actual headline to classify at the end 4. Send to the model and extract the response 5. Return prediction with token usage metrics

```

[ ]: def classify_few_shot(
    headline: str,
    examples: list[dict],
    model_name: str
) -> tuple[str, int, int]:

```

```

# System prompt for few-shot prompting
system_prompt = (

    # Define classification task
    f"Classify the input headline into one of these categories:␣
↪{VALID_CATEGORIES}. "

    # Tell model to mimic example format
    "Follow the exact format shown in the examples. "

    # Restrict output format
    "Provide ONLY the category name."
)

# Convert each example into formatted text block
example_blocks = [

    # Example format:
    # Headline: ...
    # Category: ...
    f"Headline: {ex['text']}\nCategory: {ex['label']}"

    # Iterate through all examples
    for ex in examples
]

# Combine all examples into one user prompt
# Then append actual query headline
user_content = (

    # Separate examples using blank lines
    "\n\n".join(example_blocks)

    # Add actual headline to classify
    + f"\n\nHeadline: {headline}\nCategory:"
)

# Send request to Ollama
response = ollama.chat(

    model=model_name,

    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_content}
    ],

```

```

    # Deterministic generation
    options={"temperature": 0.0}
)

# Extract model response
raw_content = response['message']['content'].strip()

# Return cleaned prediction + token counts
return (
    clean_label(raw_content),
    response.get('prompt_eval_count', 0),
    response.get('eval_count', 0)
)

```

1.6 6. Chain-of-Thought Classification

Chain-of-Thought (CoT) Prompting encourages the model to reason step-by-step BEFORE giving the final answer.

Key characteristics: - Most generated tokens (most reasoning required) - Often improves accuracy on complex tasks - Allows us to see the model's intermediate reasoning - Temperature set to 0.2 for some variety in reasoning - Useful for understanding why a classification was made

How it works: 1. Create a system prompt asking for step-by-step reasoning 2. Request output in format: "Final Category: [label]" 3. Extract the step-by-step reasoning and final category 4. Use regex to parse the final category line 5. Return prediction with token counts 6. Fallback to clean_label if the expected format isn't found

```

[ ]: def classify_cot(headline: str, model_name: str) -> tuple[str, int, int]:

    # CoT system prompt
    system_prompt = (

        # Task description
        f"Classify the news headline into one of: {VALID_CATEGORIES}. "

        # Ask the model to reason step-by-step
        "First, reason step-by-step about what fields or industries the words_
        ↪in the headline relate to. "

        # Force final answer format
        "Finally, end your response with the phrase "
        "'Final Category: <label>'."
    )

    # Query Ollama
    response = ollama.chat(

```

```

    model=model_name,

    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": f"Headline: {headline}"}
    ],

    # Slightly higher temperature to encourage reasoning
    options={"temperature": 0.2}
)

# Extract raw generated output
raw_content = response['message']['content'].strip()

# Use regex to extract:
# Final Category: <label>
final_line_match = re.search(
    r"Final Category:\s*(\w+)",
    raw_content,
    re.IGNORECASE
)

# If explicit final category exists:
if final_line_match:

    # Extract label from regex group
    label = clean_label(final_line_match.group(1))

else:
    # Otherwise try cleaning whole response
    label = clean_label(raw_content)

# Return prediction + token statistics
return (
    label,
    response.get('prompt_eval_count', 0),
    response.get('eval_count', 0)
)

```

1.7 7. Data Loading Function

The `load_data()` function handles dataset loading with graceful fallback to synthetic data.

Features: - Reads headlines from one file (one per line) - Reads corresponding labels from another file (one per line) - Validates that headline and label counts match - Falls back to synthetic dataset if files not found or not specified - Useful for both real datasets and testing/demo scenarios

Usage: - With files: `load_data("headlines.txt", "labels.txt")` - With synthetic data:

load_data(None, None) or load_data("", "")

```
[ ]: def load_data(input_file: str, labels_file: str) -> list[dict]:

    # If both files are provided
    if input_file and labels_file:

        try:

            # Open headline file
            with open(input_file, 'r', encoding='utf-8') as f_in,
            ↪open(labels_file, 'r', encoding='utf-8') as f_lbl:

                headlines = [
                    line.strip()
                    for line in f_in
                    if line.strip()
                ]

                # Read labels
                labels = [
                    line.strip()
                    for line in f_lbl
                    if line.strip()
                ]

                # Warn if counts mismatch
                if len(headlines) != len(labels):

                    print(
                        f"Warning: Mismatch in lines! "
                        f"{len(headlines)} headlines vs "
                        f"{len(labels)} labels."
                    )

                # Combine headlines and labels together
                return [
                    {"headline": h, "label": l}
                    for h, l in zip(headlines, labels)
                ]

            # Handle missing files
            except FileNotFoundError as e:

                print(f"Error loading files: {e}")

                # Exit program with error
```

```

        exit(1)

    else:

        # Fallback mode
        print(
            "No input files provided via arguments. "
            "Using built-in synthetic dataset."
        )

    return SYNTHETIC_DATASET

```

1.8 8. Run Evaluation and Compare Strategies

The `run_evaluation()` function is the main orchestrator that:

1. **Tests all three prompting strategies** on every headline in the dataset
2. **Tracks metrics** for each approach:
 - Accuracy: Percentage of correct predictions
 - Prompt tokens: How many tokens the model consumed reading the prompt
 - Generated tokens: How many tokens the model produced
 - Total tokens: Combined token usage (efficiency metric)
3. **Processes each headline through all three methods:**
 - Zero-Shot: Quick but less informed
 - Few-Shot: Balanced approach with examples
 - Chain-of-Thought: Reasoning-based approach
4. **Displays results in a formatted table** showing:
 - Correct predictions per strategy
 - Overall accuracy percentage
 - Average token usage for efficiency comparison

Output: Comprehensive comparison showing trade-offs between accuracy and computational cost

```

[ ]: def run_evaluation(dataset: list[dict], model_name: str):

    # Startup information
    print(f"\nInitializing Prompt Engineering Playground...")
    print(
        f"Model: '{model_name}' | "
        f"Total Headlines: {len(dataset)}\n"
    )

    # Dictionary to track metrics for each prompting strategy
    metrics = {

        "Zero-Shot": {
            "correct": 0,
            "prompt_tok": 0,

```

```

        "comp_tok": 0
    },

    "Few-Shot": {
        "correct": 0,
        "prompt_tok": 0,
        "comp_tok": 0
    },

    "Chain-of-Thought": {
        "correct": 0,
        "prompt_tok": 0,
        "comp_tok": 0
    }
}

# Total dataset size
total_items = len(dataset)

# Iterate through dataset
for idx, item in enumerate(dataset, 1):

    # Extract headline
    headline = item["headline"]

    # Extract true label
    ground_truth = item["label"]

    # Progress indicator
    print(f"Processing [{idx}/{total_items}]: \"{headline[:40]}...\"")

    # -----
    # ZERO-SHOT
    # -----

    # Run zero-shot classification
    zs_pred, zs_p_tok, zs_c_tok = classify_zero_shot(
        headline,
        model_name
    )

    # Accumulate token statistics
    metrics["Zero-Shot"]["prompt_tok"] += zs_p_tok
    metrics["Zero-Shot"]["comp_tok"] += zs_c_tok

    # Update accuracy count
    if zs_pred == ground_truth:

```

```

        metrics["Zero-Shot"]["correct"] += 1

# -----
# FEW-SHOT
# -----

fs_pred, fs_p_tok, fs_c_tok = classify_few_shot(
    headline,
    FEW_SHOT_EXAMPLES,
    model_name
)

metrics["Few-Shot"]["prompt_tok"] += fs_p_tok
metrics["Few-Shot"]["comp_tok"] += fs_c_tok

if fs_pred == ground_truth:
    metrics["Few-Shot"]["correct"] += 1

# -----
# CHAIN-OF-THOUGHT
# -----

cot_pred, cot_p_tok, cot_c_tok = classify_cot(
    headline,
    model_name
)

metrics["Chain-of-Thought"]["prompt_tok"] += cot_p_tok
metrics["Chain-of-Thought"]["comp_tok"] += cot_c_tok

if cot_pred == ground_truth:
    metrics["Chain-of-Thought"]["correct"] += 1

# -----
# PRINT RESULTS
# -----

print("\n" + "=" * 70)

print("                                PROMPT DESIGN EVALUATION RESULTS                                ")

print("=" * 70 + "\n")

# Data structure for tabulate
table_data = []

# Process metrics for each approach

```

```

for approach, data in metrics.items():

    # Accuracy percentage
    accuracy = (data["correct"] / total_items) * 100

    # Average prompt tokens
    avg_prompt = data["prompt_tok"] / total_items

    # Average generated tokens
    avg_comp = data["comp_tok"] / total_items

    # Total average tokens
    avg_total = avg_prompt + avg_comp

    # Add row to table
    table_data.append([
        approach,
        f"{data['correct']}/{total_items}",
        f"{accuracy:.1f}%",
        f"{avg_prompt:.1f}",
        f"{avg_comp:.1f}",
        f"{avg_total:.1f}"
    ])

# Table headers
headers = [
    "Approach",
    "Correct",
    "Accuracy",
    "Avg Prompt Tok",
    "Avg Gen Tok",
    "Avg Total Tok"
]

# Print formatted table
print(tabulate(
    table_data,
    headers=headers,
    tablefmt="grid"
))

```

1.9 9. Example Usage

This final section demonstrates how to use the classifier:

What it does: 1. Loads the synthetic dataset (5 sample headlines with labels) 2. Runs the full evaluation with the llama3 model 3. Produces a detailed comparison of all three prompting strategies

To use with your own data:

```
# Load your custom dataset
dataset = load_data("your_headlines.txt", "your_labels.txt")

# Run evaluation
run_evaluation(dataset, "llama3")
```

Requirements: - Ollama must be installed and running locally - The specified model (default: llama3) must be pulled in Ollama - The model should be able to follow instructions well for best results

Expected Output: A comparison table showing accuracy and token efficiency for each prompting strategy

```
[ ]: # Example usage: Run evaluation with synthetic dataset
if __name__ == "__main__":
    # Load dataset (uses synthetic dataset by default)
    dataset = load_data(None, None)

    # Run evaluation with llama3 model
    run_evaluation(dataset, "llama3")
```